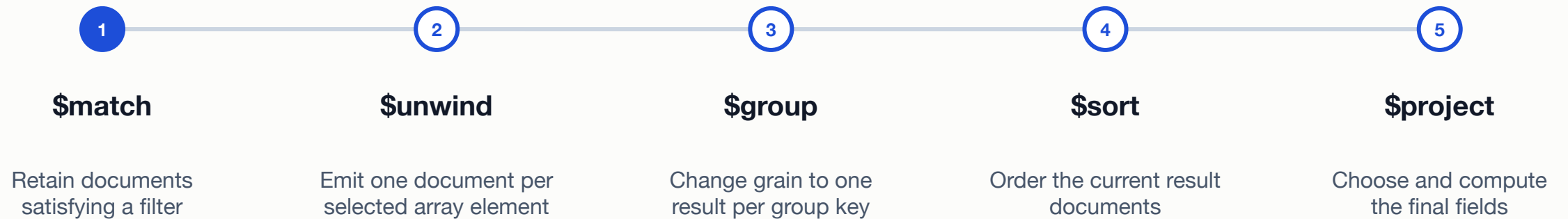


Aggregation, Validation, and MongoDB Index Evidence

Track document shape through a pipeline, reject invalid writes, and connect sort performance to a real index decision.

A pipeline is an ordered sequence of transformations

Every stage receives the previous stage's output shape and count.



Pipeline grain can change more than once

ticket → ticket-event pair → one result per category

INPUT

One source document per ticket.

AFTER UNWIND

One working document per ticket-event element.

AFTER GROUP

One output document per category with aggregate values.

Build the pipeline around one answerable workload question

```
pipeline = [  
  {"$match": {"status": {"$in": ["new", "open",  
    "in_progress"]}}},  
  {"$group": {  
    "_id": "$category",  
    "ticket_count": {"$sum": 1},  
    "highest_priority": {"$max": "$priority_rank"}  
  }},  
  {"$sort": {"ticket_count": -1, "_id": 1}},  
  {"$project": {  
    "_id": 0, "category": "$_id",  
    "ticket_count": 1, "highest_priority": 1  
  }}  
]
```

Result

- One input document per active ticket
- One grouped result per category
- Stable tie-break order by category

Move filters early only when the meaning remains the same

Helpful early match

- Uses source fields available before transformation
- Reduces documents entering expensive stages
- May allow an index to support the initial pipeline stage

Invalid movement

- Filters on a computed field before it exists
- Moves a post-group condition to source documents
- Changes which array element or group satisfies the question

Validation turns shape expectations into server behavior

A schema-flexible database can still reject missing, mistyped, or unsupported values.

REQUIRED

Fields that must exist for every accepted document.

TYPE

BSON types expected for identity, date, number, object, or array.

VALUE

Enumerations or nested rules that protect the collection contract.

A validator documents required fields, BSON types, and values

```
validator = {
  "$jsonSchema": {
    "bsonType": "object",
    "required": ["ticket_id", "status", "opened_at"],
    "properties": {
      "ticket_id": {"bsonType": ["int", "long"]},
      "status": {"enum": ["new", "open", "in_progress",
"resolved", "closed"]},
      "opened_at": {"bsonType": "date"}
    }
  }
}
```

Boundary

- BSON date is not an arbitrary date-looking string.
- Enum protects vocabulary, not legal transitions.
- Server behavior requires a real MongoDB path.

Validation needs an accepted and a rejected write

Expected success

- All required fields are present
- BSON types match the declared contract
- Status belongs to the supported vocabulary

Expected rejection

- Use one deliberately invalid field or missing requirement
- Confirm the validation failure rather than another error
- Remove or roll back any disposable test data

Lab 1: Pipeline and validation studio

Build one aggregation with tracked grain and test one document validation rule.

- Predict input and output identifiers, counts, and shape for each pipeline stage.
- Run the pipeline, verify grouped results, and explain one stage-order decision.
- Install or inspect the validator and record one valid and one expected-invalid write.

SUBMIT ONE THING / one pipeline-and-validation evidence packet

Filter, sort, and index order form one workload

A sort becomes expensive when too many documents reach it without a supporting order.

Compound index order follows the filter-and-sort pattern

Query part	Example	Candidate key	Reasoning
Equality filter	status = 'open'	{ status: 1, ... }	narrow the key range first
Second equality	category = 'streetlight'	{ ..., category: 1, ... }	continue equality prefix
Sort	opened_at descending	{ ..., opened_at: -1 }	read matching keys in requested order
Projection	ticket_id + subject	optional covered fields	wider index has greater cost

Explain connects the query, index, and observed work

Before candidate

- Collection scan or broad keys/documents examined
- Explicit sort stage or larger working set
- Result count alone hides work

After candidate

- Index bounds match the filter prefix
- Sort is supported or reduced
- Keys/documents examined and result correctness are compared

Lab 2: Sort performance and case-study response

Complete the distinct sort-stage lab, then connect the index decision to an event-driven inventory case.

- Complete Improving Performance of Sort Stages - Lab Only individually and capture redacted score evidence.
- State the filter, sort, index mechanism, and one write/storage tradeoff in your own words.
- Write the case response as 200–300 words of Brightspace text using claim, evidence, and tradeoff.

SUBMIT ONE THING / one Brightspace text response with redacted completion evidence

A career response makes a technical decision visible

- 1 Claim: state the design or operations decision directly rather than summarizing the video.

- 2 Evidence: cite a concrete workload, pipeline, validation, index, or reliability detail.

- 3 Tradeoff: identify a cost, consistency risk, operational burden, or unanswered measurement.

- 4 Professional clarity: write concise Brightspace text and avoid unsupported product praise.

MongoDB operations connect shape, rules, and measured work

Track pipeline grain, enforce the document boundary, and justify indexes from explain evidence.

- What does one document represent after each stage?

- Which invalid write does validation reject?

- Which filter-and-sort work changed after the index?